

CO3-AT1-Analytical Problem Solving

Name: V. Reddy Sreya

Regno:192525124

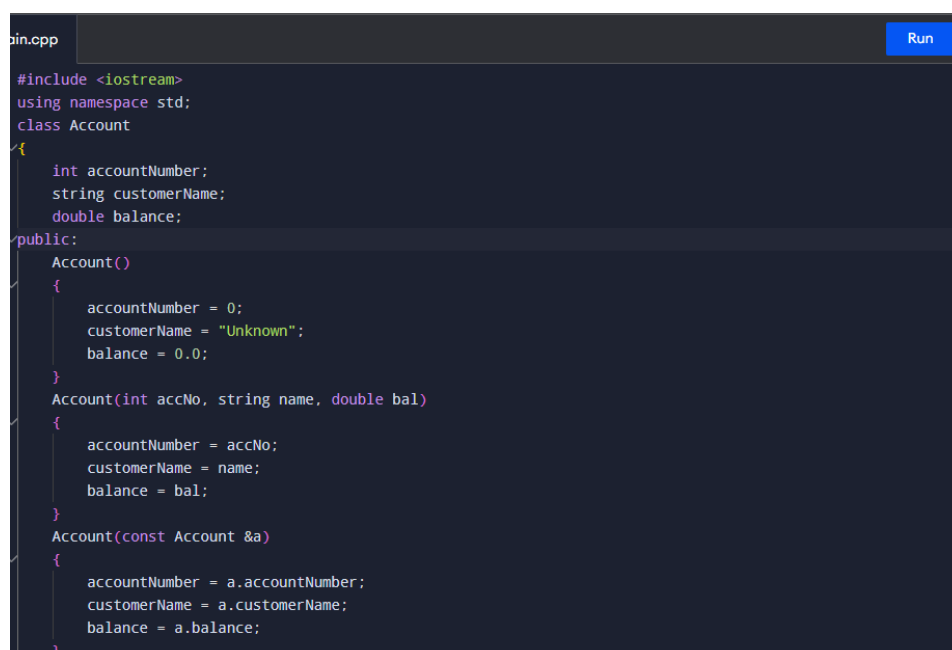
1. Banking Account Management System

A banking application needs to create and manage customer account objects. The system should support different ways of initializing an account depending on the information available. Design a C++ program using a class Account with the following constructors: Default constructor - initializes an account with default values. Parameterized constructor - initializes account number, customer name, and balance. Copy constructor - creates a new account by copying the details of an existing account.

Tasks:

1. Define the Account class with suitable data members.
2. Implement the default, parameterized, and copy constructors.
3. Create objects using each constructor.
4. Display the details of each account.
5. Analyze and explain how object initialization differs for the three constructors.
6. Create a new account object from an existing account object using the copy constructor.
7. Predict and explain the order in which constructors are called during object creation and copying.

Code:



```
bin.cpp Run
#include <iostream>
using namespace std;
class Account
{
    int accountNumber;
    string customerName;
    double balance;
public:
    Account()
    {
        accountNumber = 0;
        customerName = "Unknown";
        balance = 0.0;
    }
    Account(int accNo, string name, double bal)
    {
        accountNumber = accNo;
        customerName = name;
        balance = bal;
    }
    Account(const Account &a)
    {
        accountNumber = a.accountNumber;
        customerName = a.customerName;
        balance = a.balance;
    }
}
```

```

void display()
{
    cout << "Account Number : " << accountNumber << endl;
    cout << "Customer Name : " << customerName << endl;
    cout << "Balance : " << balance << endl;
    cout << endl;
}
};
int main()
{
    Account a1;
    Account a2(1001, "Ravi", 50000);
    Account a3(a2);
    cout << "Default Constructor:" << endl;
    a1.display();
    cout << "Parameterized Constructor:" << endl;
    a2.display();
    cout << "Copy Constructor:" << endl;
    a3.display();
    return 0;
}

```

Output:

```

Output
Default Constructor:
Account Number : 0
Customer Name : Unknown
Balance : 0

Parameterized Constructor:
Account Number : 1001
Customer Name : Ravi
Balance : 50000

Copy Constructor:
Account Number : 1001
Customer Name : Ravi
Balance : 50000

--- Code Execution Successful ---

```

2.Inventory Product Management System

A software company is developing an inventory management system using a class called Product. The system should allow products to be created, copied, combined, and removed from memory. Write a complete C++ program for the Inventory Product Management System using a parameterized constructor, copy constructor, destructor, and overloaded + operator. Demonstrate the complete object lifecycle through suitable output statements

Analyze:

1. Design the Product class with appropriate data members.
2. Implement a parameterized constructor to initialize product details.
3. Implement a copy constructor to duplicate a product.
4. Implement a destructor to release resources when an object is destroyed.
5. Overload the + operator to add the stock quantities of two products.
6. Create two product objects using the parameterized constructor.

7. Create another product using the copy constructor.
8. Add two products using the overloaded + operator.
9. Display the product details and resulting stock quantity.
10. Trace and explain the sequence of constructor and destructor calls.

Code:

```
#include <iostream>
using namespace std;
class Product
{
    int productId;
    string productName;
    int stock;
public:
    Product(int id, string name, int s)
    {
        productId = id;
        productName = name;
        stock = s;
        cout << "Parameterized Constructor called for "
              << productName << endl;
    }
    Product(const Product &p)
    {
        productId = p.productId;
        productName = p.productName;
        stock = p.stock;

        cout << "Copy Constructor called for "
              << productName << endl;
    }
    Product operator+(Product p)
    {
        Product temp(0, "Combined Product", stock + p.stock);
        return temp;
    }
}
```

```
void display()
{
    cout << "Product ID   : " << productId << endl;
    cout << "Product Name : " << productName << endl;
    cout << "Stock       : " << stock << endl;
    cout << endl;
}

~Product()
{
    cout << "Destructor called for "
          << productName << endl;
}

int main()
{
    cout << "Creating Product 1:" << endl;
    Product p1(101, "Laptop", 50);
    cout << "\nCreating Product 2:" << endl;
    Product p2(102, "Mouse", 30);
    cout << "\nCreating Product 3 using Copy Constructor:" << endl;
    Product p3(p1);
    cout << "\nProduct 1 Details:" << endl;
    p1.display();
    cout << "Product 2 Details:" << endl;
    p2.display();
    cout << "Copied Product Details:" << endl;
    p3.display();
    cout << "Adding Product 1 and Product 2:" << endl;
    Product p4 = p1 + p2;
```

```

    cout << "\nCombined Product Details:" << endl;
    p4.display();
    cout << "End of program." << endl;

    return 0;
}

```

Output:

```

Output
Creating Product 1:
Parameterized Constructor called for Laptop

Creating Product 2:
Parameterized Constructor called for Mouse

Creating Product 3 using Copy Constructor:
Copy Constructor called for Laptop

Product 1 Details:
Product ID : 101
Product Name : Laptop
Stock : 50

Product 2 Details:
Product ID : 102
Product Name : Mouse
Stock : 30

Copied Product Details:
Product ID : 101
Product Name : Laptop
Stock : 50

Adding Product 1 and Product 2:
Copy Constructor called for Mouse
Parameterized Constructor called for Combined Product
Destructor called for Mouse

Combined Product Details:
Product ID : 0
Product Name : Combined Product
Stock : 80

End of program.
Destructor called for Combined Product
Destructor called for Laptop
Destructor called for Mouse
Destructor called for Laptop

=== Code Execution Successful ===

```

3. Question 3: Shopping Cart Management System

An online shopping application maintains customer shopping carts. A cart contains multiple products and their quantities. The system should support creating a cart, duplicating an existing cart, and merging two carts. Write a complete C++ program to implement the Shopping Cart system. Demonstrate parameterized construction, copy construction, cart merging using +, and destruction of objects.

Tasks:

1. Design the Product class with appropriate data members.
2. Implement a parameterized constructor to initialize product details.

3. Implement a copy constructor to duplicate a product.
4. Implement a destructor to release resources when an object is destroyed.
5. Overload the + operator to add the stock quantities of two products.
6. Create two product objects using the parameterized constructor.
7. Create another product using the copy constructor.
8. Add two products using the overloaded operator.
9. Display the product details and resulting stock quantity.
10. Trace and explain the sequence of constructor and destructor calls.
11. Explain what may happen if dynamically allocated memory is not released in the destructor

Code:

```
#include <iostream>
using namespace std;
class ShoppingCart
{
    int productId;
    string productName;
    int quantity;

public:
    ShoppingCart(int id, string name, int q)
    {
        productId = id;
        productName = name;
        quantity = q;
        cout << "Parameterized Constructor called for "
              << productName << endl;
    }
    ShoppingCart(const ShoppingCart &c)
    {
        productId = c.productId;
        productName = c.productName;
        quantity = c.quantity;
        cout << "Copy Constructor called for "
              << productName << endl;
    }
    ShoppingCart operator+(ShoppingCart c)
    {
        ShoppingCart temp(0, "Merged Cart", quantity + c.quantity);
        return temp;
    }
}
```

```

void display()
{
    cout << "Product ID   : " << productId << endl;
    cout << "Product Name : " << productName << endl;
    cout << "Quantity      : " << quantity << endl;
    cout << endl;
}

~ShoppingCart()
{
    cout << "Destructor called for "
        << productName << endl;
}

int main()

    cout << "Creating Cart 1:" << endl;
    ShoppingCart c1(101, "Laptop", 2);
    cout << "\nCreating Cart 2:" << endl;
    ShoppingCart c2(102, "Mouse", 5);
    cout << "\nCreating Cart 3 using Copy Constructor:" << endl;
    ShoppingCart c3(c1);
    cout << "\nCart 1 Details:" << endl;
    c1.display();
    cout << "Cart 2 Details:" << endl;
    c2.display();
    cout << "Copied Cart Details:" << endl;
    c3.display();
    cout << "Merging Cart 1 and Cart 2:" << endl;
    ShoppingCart c4 = c1 + c2;

```

```

    cout << "\nMerged Cart Details:" << endl;
    c4.display();
    cout << "End of program." << endl;
    return 0;
}

```

Output:

```

Output
Creating Cart 1:
Parameterized Constructor called for Laptop

Creating Cart 2:
Parameterized Constructor called for Mouse

Creating Cart 3 using Copy Constructor:
Copy Constructor called for Laptop

Cart 1 Details:
Product ID   : 101
Product Name : Laptop
Quantity      : 2

Cart 2 Details:
Product ID   : 102
Product Name : Mouse
Quantity      : 5

Copied Cart Details:
Product ID   : 101
Product Name : Laptop
Quantity      : 2

Merging Cart 1 and Cart 2:
Copy Constructor called for Mouse
Parameterized Constructor called for Merged Cart
Destructor called for Mouse

Merged Cart Details:
Product ID   : 0
Product Name : Merged Cart
Quantity      : 7

End of program.
Destructor called for Merged Cart
Destructor called for Laptop
Destructor called for Mouse
Destructor called for Laptop

```

=== Code Execution Successful ===

